Robotics 24-25
Group 11

Ismael Díaz Romero
Uex Politécnica
Cáceres, España
isdiazr@alumnos.unex.es

José Javier Pérez Sánchez
Uex Politécnica
Cáceres, España
jperezeb@alumnos.unex.es

Sergio Jiménez Macías
Uex Politécnica
Cáceres, España
sjimenezr@alumnos.unex.es

***Abstract*—This paper presents a robust robotic navigation system designed for dynamic and cluttered environments. The proposed system integrates three core components: a grid-based environment representation, a path-planning module, and a motion controller. A 2D grid, dynamically updated using LiDAR sensor data, models the environment and provides the foundation for optimal path computation using Dijkstra's algorithm. The motion controller translates planned paths into smooth and reliable motion commands, ensuring real-time adaptability and convergence to the target.**

Key challenges encountered during the development include concurrency issues, unreachable paths, dynamic obstacles, and path ambiguity in obstacle avoidance. Concurrency challenges were addressed using mutexes and double buffering to ensure consistent grid updates and path-planning requests. Fail-safe mechanisms handled scenarios with no available path, while obstacle expansion techniques enabled dynamic obstacle avoidance. However, the system struggled with path ambiguity, highlighting the need for enhanced decision-making capabilities.

This work demonstrates the effectiveness of modular, grid-based navigation systems in handling real-time navigation tasks. Future research will explore extending the system to 3D environments, integrating advanced path-planning algorithms, and incorporating learning-based enhancements for adaptive decision-making.

I. Introduction

A. Introduction to Robotics

Robotics is an interdisciplinary field combining elements of engineering, artificial intelligence (AI), and computer science to create systems capable of performing tasks autonomously or with limited human intervention. Over the past few decades, robotics has evolved from simple automated systems for manufacturing to complex, adaptive systems capable of operating in unstructured and dynamic environments. This evolution has been driven by both engineering and AI approaches, each contributing distinct methodologies and applications.

The early development of robotics largely centered on manufacturing and industrial applications. In these settings, robots—often referred to as "industrial manipulators"—were programmed for high-precision, repetitive tasks on structured assembly lines, where consistency and repeatability were paramount. However, these early systems typically operated in confined environments, relying on rigid programming to complete narrowly defined tasks. This limited their adaptability and environmental awareness, rendering them unsuitable for applications beyond the controlled confines of industrial facilities.

The shift toward AI-enhanced robotics introduced new capabilities, marking a significant departure from traditional engineering approaches. AI-enabled robots incorporate sensory systems and autonomous decision-making capabilities, allowing them to operate in unpredictable environments. A notable application of AI robotics is space exploration, where autonomous planetary rovers, such as those developed by NASA, must navigate and adapt to unknown terrains without continuous human oversight. These systems demonstrate the potential of AI to extend robotic functionality beyond mere automation, toward autonomy and real-time environmental interaction.

The versatility of robots extends far beyond industrial settings and includes applications in healthcare, exploration, and environmental monitoring. Robots such as surgical systems have revolutionized modern medicine by enabling minimally invasive procedures with enhanced precision and reduced patient recovery time. Exoskeletons, on the other hand, are wearable robots designed to augment human strength, assisting workers in physically demanding environments or aiding individuals with mobility impairments. Furthermore, robots are increasingly used to address critical global challenges, such as inspecting infrastructure, monitoring natural resources, and mitigating environmental degradation.

Mobile robots, a key area within the field, involve robots that navigate and interact with their surroundings, often using varied locomotion modes, such as wheels, tracks, or thrusters, to move over land, through water, or even in the air. Mobile robots equipped with advanced sensors like LiDAR, sonar, and cameras are deployed in hazardous environments, from deep-sea exploration to monitoring the interior of volcanoes. In logistics, robots automate the transportation of goods in warehouses and ports, while in agriculture, drones and autonomous vehicles are transforming food production through precise monitoring of crops and efficient resource management. These applications illustrate the adaptability of robots to different terrains and operational requirements.

The concept of autonomy in robotics has become increasingly important in recent years. Modern autonomous robots

combine sensor data with real-time processing capabilities to navigate and perform complex tasks independently. Self-driving cars, for instance, rely on an array of sensors and algorithms to detect obstacles, make navigational decisions, and adapt to dynamic traffic conditions. Autonomy has also been crucial in robots designed for planetary exploration. Mars rovers, such as Spirit, Opportunity, and Curiosity, have demonstrated unparalleled capabilities in analyzing the Martian surface, collecting valuable scientific data while autonomously adapting to challenging terrains. These missions highlight the potential of robotics to extend human presence and knowledge into environments that are otherwise inaccessible.

Another emerging field in robotics is the integration of human-like features and behaviors into humanoid robots. Robots like Honda's Asimo or Chinese ping-pong-playing robots exemplify advancements in mimicking human movements, hand-eye coordination, and problem-solving abilities. These humanoid systems aim to create machines that not only resemble humans in appearance but also perform tasks requiring fine motor skills, such as manipulation of delicate objects. Although significant progress has been made, challenges remain in integrating these components into reliable and cost-effective systems for widespread use.

Environmental robotics has gained prominence in addressing planetary-scale challenges. Robots equipped with sensors for chemical analysis and imaging technologies are used to monitor the health of ecosystems, such as coral reefs and rainforests. Additionally, aerial and underwater robots are deployed to study climate phenomena, such as hurricanes and ocean currents, and to ensure the sustainability of natural resources. These efforts demonstrate how robots contribute to understanding and preserving the planet's health.

Robotics is not only a key enabler of current technological progress but also an essential component for addressing global challenges. Its applications range from improving healthcare accessibility and efficiency to monitoring and preserving the planet's health, managing natural resources, and ensuring the functionality of critical infrastructures. Moreover, robots continue to take on tasks characterized as "dirty, dull, and dangerous," such as disaster response, nuclear decommissioning, and industrial inspections, ensuring the safety of human workers while maintaining efficiency. As robotics evolves, its integration into daily life and industries will redefine the boundaries of human capability and innovation.

B. Component-Based Software Engineering

Component-Based Software Engineering (CBSE) is a methodology foundational to developing complex robotic systems, prioritizing modularity, reusability, and adaptability to diverse application needs. By segmenting software into components—each responsible for a function like path planning or collision detection—CBSE promotes reliable, maintainable, and flexible designs. Components are defined with standardized interfaces and dependencies, enabling them to function independently and interchangeably. This modular structure is crucial in robotics, where real-time processing, dependability, and adaptability to dynamic environments are key.

In robotics, CBSE supports reusability through horizontal and vertical components. Horizontal components provide general functions, such as hardware interfacing, usable across applications, while vertical components address domain-specific tasks like navigation or sensor processing. This structure allows robotic systems to adapt to new requirements and achieve consistent performance, supporting efficient development and modular integration across different applications.

1) Modularity and System Architecture Principles in CBSE: CBSE emphasizes separating specification from implementation, ensuring components are interoperable and adaptable across diverse systems. This separation allows components to evolve independently as long as interfaces remain consistent, which is essential in robotics, where frequent hardware updates require flexible integration. Standardized interfaces further support the integration of third-party modules and maintain core architecture integrity.

CBSE organizes functionality into four main modules:

Computation: Manages data processing tasks, such as sensor data handling and motion planning. Configuration: Controls system assembly, managing active components and connections. Communication: Facilitates data exchange across components, supporting synchronous and asynchronous flows. Coordination: Synchronizes components, critical for real-time operations. This modular approach allows isolated adjustments and enhancements, promoting flexibility and scalability to meet evolving performance demands.

2) Hardware Variability and System Scalability: CBSE addresses challenges posed by varied hardware and operational needs in robotics. Robotic systems incorporate diverse sensors and actuators, requiring adaptable software across centralized or distributed architectures. CBSE's modularity enables these systems to scale and reconfigure seamlessly, as components are designed with minimal hardware dependencies, allowing easy integration or replacement.

3) Frameworks and Practical Applications: Frameworks like Open Robot Control Software (OROCOS) and Open Robot Control Architecture (ORCA) bring CBSE principles to robotics, simplifying component development and integration. OROCOS provides real-time control through modular components for key functions, while ORCA automates configuration, allowing rapid deployment and reconfiguration in dynamic settings. These frameworks exemplify CBSE's ability to support modular, adaptable robotic architectures, enabling systems to operate reliably amid changing conditions.

4) Conclusion: CBSE offers a structured, flexible approach to developing complex robotic systems, enhancing modularity, reusability, and adaptability. Frameworks like OROCOS and ORCA showcase CBSE's role in advancing robotics, supporting real-time performance, distributed processing, and efficient

resource allocation, making it a valuable foundation for future autonomous systems.

II. Materials and methods

To carry out this robotics practice, we used the lab computers, with our specific machine featuring the following hardware specifications:

Processor: AMD Ryzen 7, later changed for a Intel i9 Graphics card: Nvidia RTX 3060 RAM: 16 GB These hardware features provided a solid foundation for running the required simulation and development software, as processing power and memory are crucial for advanced robotic simulation. The AMD Ryzen 5 processor, with its multi-core capabilities and high performance, facilitated simultaneous execution of intensive simulation and compilation tasks. The Nvidia RTX 3060 graphics card, with high-resolution graphics processing, enabled 3D simulations to run smoothly, offering a realistic, high-fidelity environment for developing and testing perception algorithms.

For software development, we used two key programming and simulation environments: CLion and WeBots.

CLion is an integrated development environment (IDE) created by JetBrains, specifically designed for C++ programming. CLion offers advanced code analysis and real-time debugging capabilities, which are essential for software development in robotics, where complex algorithms and efficient resource control are needed. Through CMake, CLion enables automated compilation, facilitating the organization and maintenance of large code projects.

Additionally, we used C++23, the latest version of the language, whose new features integrate optimally with CLion. C++23 introduces significant improvements in performance, memory management, and generic programming, all of which are enhanced and supported by CLion's tools. For example, C++23 includes "range adaptors" and "views," which allow for more flexible and efficient data structure operations. CLion leverages these capabilities through its analysis and autocompletion systems, suggesting improvements in the use of these new functions and helping detect errors in real-time. These features are especially valuable in robotics, as they allow for more effective manipulation of the large data volumes generated by sensors.

One of the standout innovations of C++23 is its initial support for "reflection," allowing developers to obtain information about the code itself during compilation. This feature, combined with CLion's real-time code analysis, makes it easier to create generic code that dynamically adapts to different data types, making robotic control systems more robust and flexible. Together, C++23 and CLion allowed us to improve productivity, optimize code performance, and reduce errors, which are crucial aspects in advanced robotics projects.

WeBots, on the other hand, is an advanced simulation platform aimed at the research and development of robotic systems in a 3D environment. This simulator is highly useful for designing and evaluating control, navigation, and perception algorithms, as it allows testing robotic behaviors in a safe virtual environment, preventing possible damage to physical components during testing phases. WeBots includes a vast library of robot, sensor, and actuator models that emulate real device behavior in various environments. This fidelity is particularly useful for testing complex interactions between the robot and its surroundings, simulating variable environmental conditions, and evaluating control algorithm robustness. WeBots supports languages such as C++ and Python and integrates with robotics control frameworks like ROS (Robot Operating System), facilitating autonomous behavior design and iterative development of robotic systems.

The operating system used on our team's computer was Ubuntu 22.04, a Linux distribution widely adopted in robotics and scientific research. Ubuntu is known for its stability, security, and compatibility with a variety of open-source development tools, essential in the robotics field. Its efficiency in resource management and ability to handle real-time applications make Ubuntu a reliable choice for running simulations and controlling robots. Additionally, its large user and developer community ensures good documentation and continuous support, which makes problem-solving and technology integration easier.

As part of the development, we also used RoboComp, a modular software infrastructure based on components for programming control architectures in robotics. RoboComp is designed to facilitate the development of distributed systems in a modular way, allowing each component of the robotic architecture to function as an independent module. This modular design is highly advantageous in robotics, as it allows the system to scale, reuse modules, and simplify maintenance. Moreover, RoboComp's modularity facilitates teamwork, enabling different team members to develop and test components independently and then integrate them seamlessly into the overall system.

In the simulation, a robot is depicted, representing the robot in the room with an approximate height of 150 cm and a width of 50 cm. The robot is equipped with several components, including a camera, two lasers, a 3D Lidar sensor, a Helios sensor, and a YOLO pattern detector, a neural network capable of identifying various objects.

For version control and project collaboration, we used GitHub. This version control platform, based on Git, allows centralized code synchronization and management, facilitating collaborative work through review and change management tools. GitHub allows collaborators to maintain a clear, detailed record of modifications, synchronize changes, identify code conflicts, and enable continuous review, which is especially useful in complex robotics projects, where code precision and coherence are essential for system functionality.

To write the technical report (paper), we used Overleaf with the IEEE Conference format standard. Overleaf is an online platform for creating LaTeX documents that enables real-time collaboration, making it easier to work as a team on academic and technical writing. The platform includes predesigned templates and automatic formatting tools, ensuring consistency and professionalism in document presentation.

The IEEE Conference standard is widely used in engineering and technology conferences, providing a standardized format that facilitates readability and comprehension of technical content, ensuring that the paper meets formal academic quality requirements.

III. Activity 1

A. *Problem Description*

The problem presented involves designing a robot similar to a Roomba, tasked with navigating various environments in the "Webots" simulator as efficiently as possible. The objective is to achieve the maximum possible percentage of surface area coverage within a 3-minute time limit. A program called "aspirator" automatically collects the results achieved by the robot.

B. *Implemented Logic Description*

The navigation architecture of the Chocachoca robot is based on a finite state machine, a structure that enables the division of robot behavior into clearly defined states. Each state activates based on sensor data and environmental context. The objective of this logic is to ensure that the robot can navigate autonomously, efficiently detecting and avoiding obstacles in a variety of scenarios. Below is a detailed description of each operational state and the transitions between them.

C. *State Structure of the Navigation System*

*1) **Forward State (FORWARD):*** In this state, the robot moves along a straight trajectory at a constant speed, allowing it to steadily cover the navigation area. This state is essential for basic movement and represents the initial point of the navigation system, as the robot only advances in this configuration if the environment is safe and free of obstacles. During this phase, proximity sensors (e.g., LIDAR) continuously scan the frontal area for objects, and any detected signal within a predefined range triggers a state change to initiate an avoidance maneuver.

• **State Transition:** If an object is detected within the proximity range (i.e., at a distance below the safety threshold defined as `STOP_THRESHOLD`), the robot transitions to the TURN state, initiating a turning maneuver to avoid collision.

*2) **Turn State (TURN):*** The TURN state is activated in direct response to an obstacle detected along the robot's path. The turning logic considers the position and angle of the detected obstacle, as identified by the LIDAR. Using this data, the robot calculates the most appropriate direction and angle of rotation to avoid a collision, adjusting its heading and rotation speed accordingly. In this state, the robot turns in a predefined direction until it determines that the path is clear.

• **State Transition:** Once the robot finds a direction free of obstacles at a safe distance (greater than `ADVANCE_THRESHOLD`), the TURN state ends, and the system transitions to either FORWARD, FOLLOW WALL, or SPIRAL, chosen alternately based on a random value.

*3) **Spiral State (SPIRAL):*** In this state, the robot follows a spiral movement pattern to progressively and efficiently cover a larger area. The SPIRAL algorithm dynamically adjusts the forward and rotational speeds based on the proximity of obstacles detected by the LIDAR. This logic allows the robot to broadly explore open areas, while any detected obstacle results in an interruption of the spiral pattern, triggering a state change. Initially, the robot starts with a low forward speed (`spiral_adv_speed`) that gradually increases in increments of 10 units until reaching the maximum allowed speed (`MAX_ADV_SPEED`). Simultaneously, the rotation speed (`spiral_rot_speed`) decreases progressively, creating a spiral movement of increasing amplitude. This pattern ensures thorough exploration without the robot getting trapped in small cycles.

Considering the forward speed X and the rotational speed Y, they progress during the execution of the program as follows:

$$X_i = 100\,\text{mm/s} \tag{1}$$

$$X = X + 10 \tag{2}$$

$$Y_i = 1\,\text{rad/s} \tag{3}$$

$$Y = Y - 0.001 \tag{4}$$

• **Direction change:** The spiral direction is adjusted based on the position of the closest detected obstacle. If the obstacle is close to the center of the robot's frontal field of view (±0.1 radians), the algorithm randomly selects a new rotation direction (left or right). If the obstacle is located at a distant angle from the center, the rotation direction is automatically adjusted to guide the robot away from potential collision.

• **Cycle reset:** A counter (`cont`) controls the duration of the SPIRAL state. After a specific number of cycles (seven in this case), the pattern resets, restoring the forward and rotation speeds to their initial values. This ensures continued exploration without allowing the robot to reach uncontrolled speeds.

• **State Transition:** If the LIDAR detects an obstacle within a distance below a threshold (`STOP_THRESHOLD`), the state automatically changes to TURN, halting the spiral movement to perform an avoidance maneuver. If no valid LIDAR readings are available, the system transitions to FORWARD, aiming to resume straightforward movement while recalibrating sensory data.

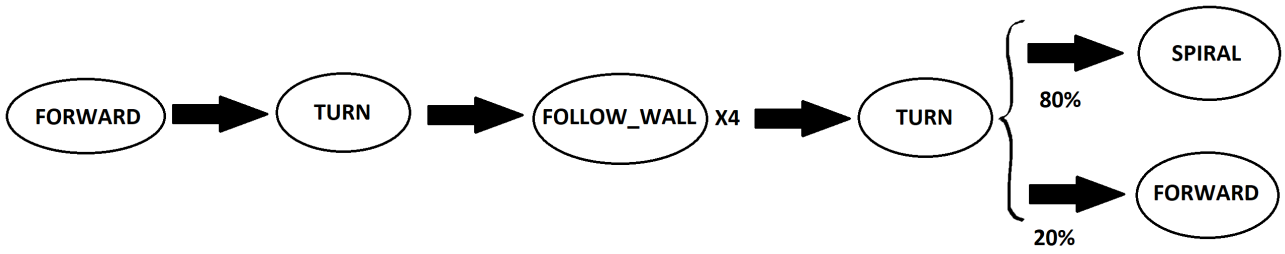


Fig. 1. State machine logic

*4) **Wall Following State (FOLLOW WALL):*** In this state, the robot maintains a safe distance from a wall or lateral obstacle, using data provided by the LIDAR at a 30-degree angle from the frontal direction. This wall-following strategy enables the robot to navigate parallel to the wall, constantly adjusting its heading to avoid collisions or getting too close to the obstacle. This state is ideal for maneuvering through narrow spaces or hallways, where the robot must maintain a reference distance (`REFERENCE_DISTANCE`) to ensure safe navigation.

The wall-following logic focuses on making small directional and speed adjustments based on the distance from the lateral obstacle. The robot modifies its orientation each time a deviation in distance is detected, allowing it to maintain a parallel trajectory.

• **State Transition:** If the robot drifts too far from the wall or if the obstacle is no longer within the direction of the sensor, the system transitions to FORWARD or SPIRAL, resuming environment exploration. This transition is guaranteed when no walls are detected within proximity for an extended period.

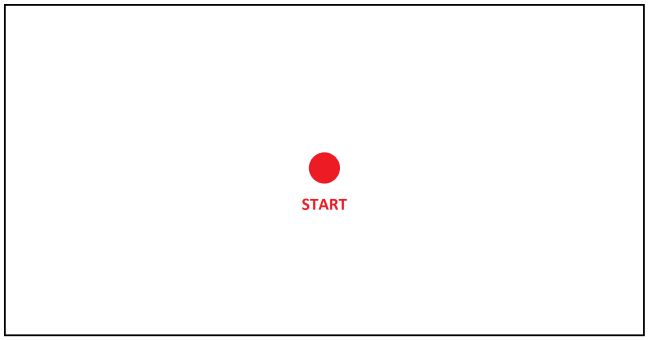


Fig. 2. Representation of the first scenario.

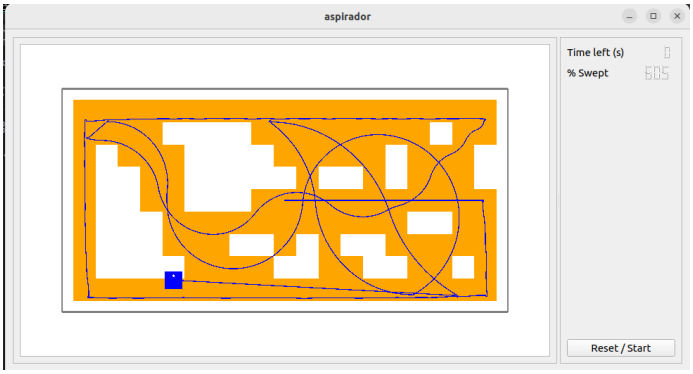


Fig. 3. Execution of the first scenario.

D. Graphical Representation and Test Scenario Results

Each of these states has been evaluated in four test scenarios, with visual analyses of the results included:

1) Scenario 1: In this scenario, the robot achieved the highest performance among all simulations conducted. The 'SPIRAL' state proved to be highly effective in wide and unobstructed environments.

2) Scenario 2: In this scenario, due to the code logic and the functionality of the 'SPIRAL' movement, the robot's entry into lateral rooms is largely dependent on chance. In several simulations, the robot remained in the center of the environment without entering any rooms. However, in cases where it did access the lateral rooms, its performance was notably effective.

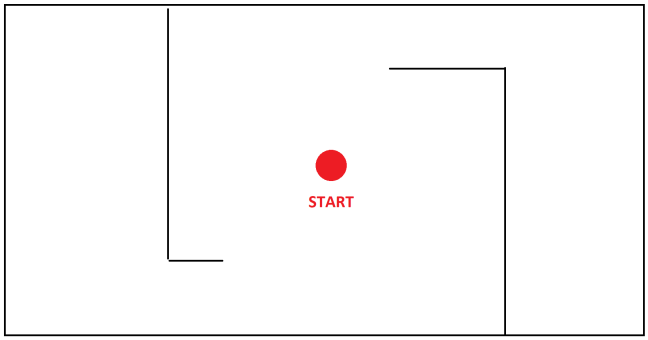


Fig. 4. Representation of the second scenario.

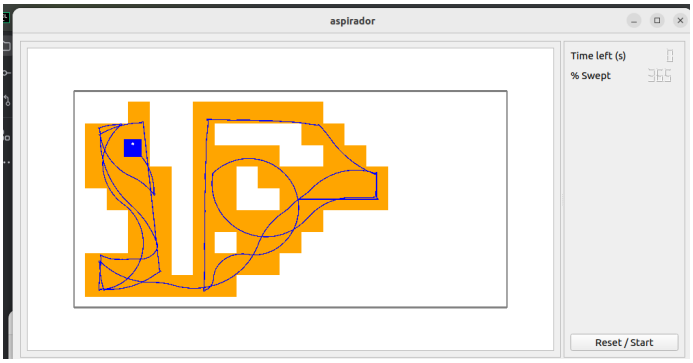


Fig. 5. Execution of the second scenario.

3) Scenario 3: In this scenario, the robot achieved the lowest performance among all tests. The robot encountered multiple issues in cleaning the designated surface, primarily failing to access the inner areas of the room. Consequently, the test yielded a low score. However, it successfully covered the perimeter of the room with high accuracy.

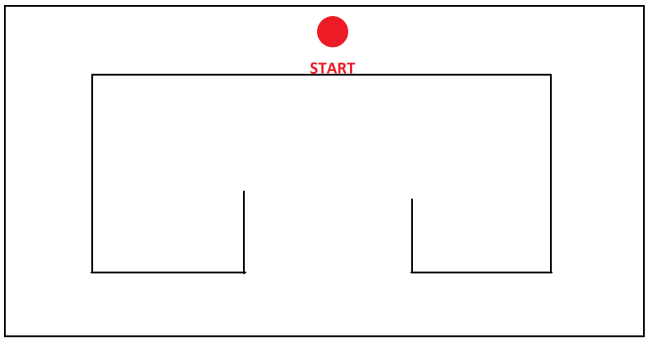


Fig. 6. Representation of the third scenario.

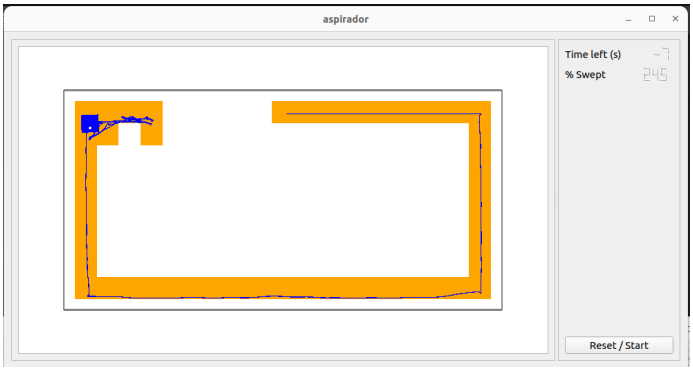


Fig. 7. Execution of the third scenario.

4) Scenario 4: In this scenario, the robot demonstrated strong overall performance. However, a higher score was not achieved due to the robot becoming stuck twice in the lower area of the room, caused by issues in both the turn and spiral methods. The turn method occasionally fails to select the optimal direction, while the spiral method does not allocate enough time to reorient around centrally positioned objects, resulting in a loop that only terminates when the spiral resets.

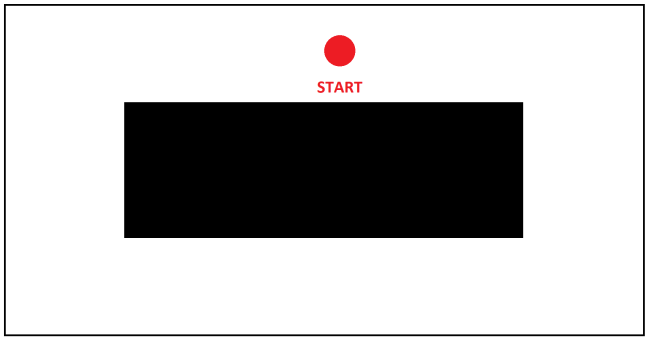


Fig. 8. Representation of the fourth scenario.

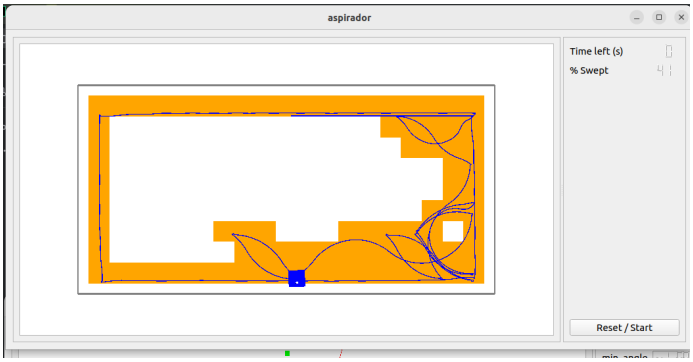


Fig. 9. Execution of the fourth scenario.

5) Results Table:

Scenarios	Results
1	60.5%
2	36.5%
3	24.5%
4	41%

IV. Activity 2: Tracking a Person using LiDAR, 360° Camera, and YOLO DNN

A. Problem Description

The objective of this activity is to develop a "Person Tracker" robot capable of following a person in a simulated environment using the "Webots" platform. The robot must track a designated target while avoiding obstacles, particularly the walls of a two-room environment. To evaluate tracking accuracy, the robot is required to maintain an optimal distance from the target and smoothly follow it for ten consecutive laps around the room without collisions.

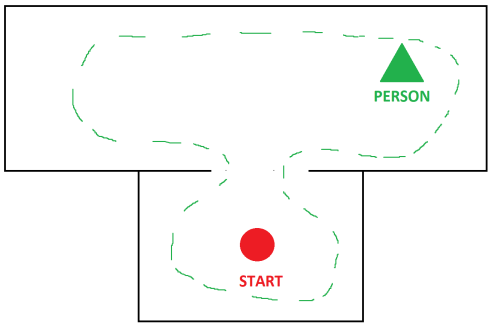


Fig. 10. Representation of the scenario.

B. System Components

To accomplish this task, several components were added to the robot's code base:

1) LiDAR Sensors: The robot is equipped with two 3D LiDARs. The bpearl LiDAR, positioned below the tray and oriented downwards, enables detection of the room's layout and obstacles. The helios LiDAR, placed at the robot's top, is used to project depth data on the camera feed, providing a 3D mapping of the surroundings.

2) 360° Camera: The Ricoh Omni camera, similar to a GoPro, offers a complete view of the environment, crucial for detecting the person's position relative to the robot.

3) YOLO DNN: The YOLO (You Only Look Once) deep neural network, implemented via the Ultralytics YOLOv11 model, serves as the primary object detection algorithm. YOLO's fast processing and high accuracy make it suitable for real-time tracking. The YOLO model is used to identify and locate the target person within the environment, relaying detection data to the robot's compute thread for responsive movement adjustments.

4) State Machine: When the target is not detected, the robot enters a SEARCH state rotating in place trying to find the person.

C. Tracking and Control Mechanism

In the compute method, the tracking algorithm calculates the relative angle between the robot and the person. Using the robot-to-person vector, the angle with the robot's nose (front orientation) is determined using the `atan2` function. We use the angle obtained from the arctangent as the rotational speed to achieve a smooth rotational movement toward the target.

$$\arctan\theta = \frac{y}{x} \tag{5}$$

The angle represents the angle relative to the person, where "x" is the horizontal distance to the person and "y" is the vertical distance to the person.

To prevent oscillations, a lineal braking function is applied, modulating advance speed based on the distance to the person being tracked. This function reduces the advance speed when the robot is getting closer to the person, allowing precise adjustments as it follows the target.

$$y = 0.55x \tag{6}$$

In this equation, "y" represents the forward speed, and "x" represents the distance to the person.

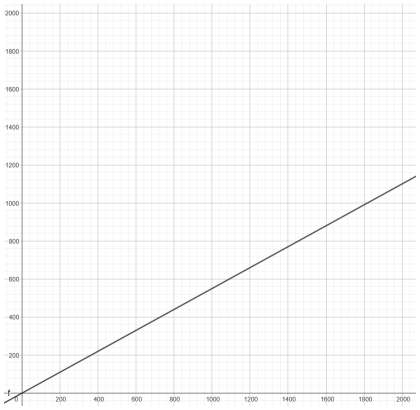


Fig. 11. Graphical representation of the linear progression of the robot's speed relative to the distance to the person.

D. Data Flow and Component Integration

The robot's tracking functionality depends on the integration of multiple data sources and processes. The YOLO DNN component publishes detection results through the VisualElementsPub interface, providing real-time location data for the target person. This data is received by the compute thread using a DoubleBuffer to manage asynchronous data flows, ensuring synchronization without blocking other processes.

Within the compute method, the state machine continuously processes input data to adjust the robot's actions. New states were introduced, including:

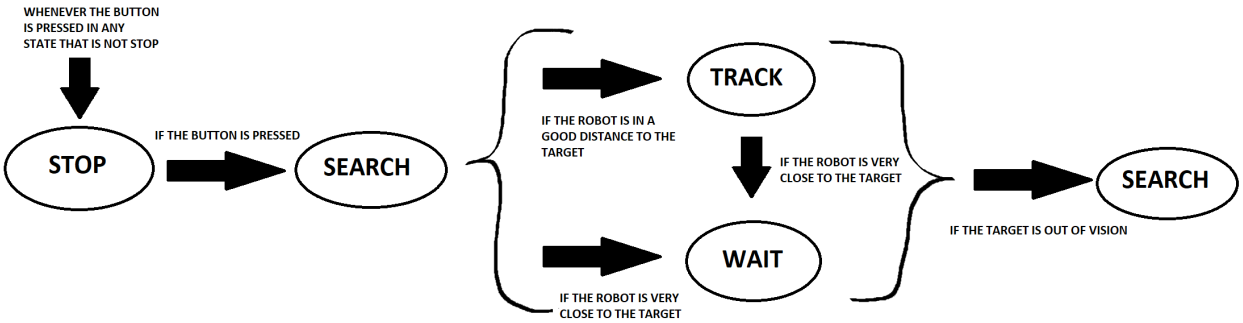


Fig. 12. Representation of the state machine.

1) TRACK: This state activates when the robot has identified a target (a person in this context). The robot uses sensory data to calculate the target's position and adjusts its forward and rotational speeds to maintain a specific distance as specified before in the document. The robot also tracks the person's movements to follow them effectively. If obstacles are detected within a threshold distance, the robot may transition to the TURN state.

2) WAIT: The WAIT state is triggered when the robot is close to the target but needs to pause due to proximity constraints. Here, the robot halts forward movement and continues to monitor the target's distance. If the target moves away beyond a set threshold, the robot may transition back to the TRACK state.

3) STOP: In the STOP state, the robot ceases all movement, commonly activated by user input or emergency conditions. This state ensures the robot remains stationary until a signal (such as releasing a stop button) allows it to transition to the TRACK state.

4) SEARCH: This state initiates when the robot loses track of the target or is unable to detect it. In SEARCH, the robot performs a rotational scan, typically turning in place to locate the target within the environment. If the target is found, the robot transitions back to the TRACK state.

5) TURN: The TURN state occurs when an obstacle is detected within the robot's path while tracking the target. The robot stops forward movement and adjusts its rotation to avoid the obstacle. Once clear, the robot returns to the TRACK state or continues its movement.

E. Results and Performance Evaluation

The robot's performance was evaluated based on the smoothness of movement throughout the tracking task. After adjustments to gain values and the braking function, the robot successfully completed ten laps following the person without collisions. The velocity control allowed rapid but stable response to the target's position changes, maintaining a consistent following distance and minimizing drift.

F. Conclusion

This task demonstrated the effectiveness of CBSE principles in integrating complex components such as LiDAR and 360° cameras for real-time person tracking. The successful implementation of a velocity controller, combined with a modular state machine, underscores the adaptability of CBSE in robotic applications. Future improvements may include refining the braking function or exploring more advanced control methods to enhance tracking stability further.

V. Activity 3: Avoiding obstacles while following Eva

A. Problem Description

The objective of this work is to develop and implement a control algorithm enabling a robot to simultaneously follow a moving target (referred to as "Eva") while avoiding obstacles in its environment.

The task builds upon the behavior implemented in the prior activity, where the robot successfully followed the target in an obstacle-free environment. In this activity, the presence of obstacles introduces additional complexity, requiring the robot to detect and avoid static objects while maintaining a safe and consistent distance from the target.

The solution must address the following key challenges:

Obstacle Detection and Representation: The robot must process LiDAR data to identify obstacles, represented as polygons. This requires clustering techniques, such as DBSCAN, to extract meaningful structures from raw point clouds while filtering irrelevant data.

1) Path Planning: Once obstacles are identified, the robot must construct a free-space representation using Delaunay triangulation and Voronoi diagrams. This representation facilitates the computation of an optimal collision-free path to the target using algorithms such as Dijkstra's.

2) Dynamic Target Following: The robot must integrate YOLO object detection data to continuously track the target's position. It must exclude the target from obstacle representations to ensure accurate path planning.

3) Control Optimization: The robot must execute the planned path using a proportional-derivative (PD) controller while maintaining minimal deviation from the target's trajectory. Adjusting control parameters dynamically to minimize deviation error is critical for performance optimization.

The task involves designing modular and reusable software components to achieve these goals, implementing robust algorithms for obstacle detection and path planning, and ensuring seamless integration with the robot's existing control system. The approach aims to provide a framework for efficient robot navigation in complex environments, addressing both real-time decision-making and computational efficiency.

B. Line extraction

The algorithm `SpecificWorker::detect_wall_lines` identifies straight wall lines from a set of input 2D points, such as those acquired from LiDAR sensors, by leveraging a Hough Transform-based feature extraction method. Below is a detailed description of the algorithm.

1) Input:

- **points**: A set of 2D points $\mathbf{P} = \{\mathbf{p}_i \in \mathbb{R}^2 : \mathbf{p}_i = (x_i, y_i)\}$, $i = 1, \ldots, N$, representing the detected points in the environment.
- **scene**: A pointer to a graphical scene object (`QGraphicsScene*`) used for visualization.

2) Output: A list of lines $\mathbf{L} = \{\mathbf{l}_j\}$, where each line $\mathbf{l}_j \in$ `QLineF` corresponds to a detected wall segment.

3) Algorithm Steps: The algorithm uses the `RoomDetector::compute_features` method to extract geometric features, including long, straight lines, from the point set $\mathbf{P}$. The process involves the following steps:

Hough Transform: The algorithm applies the **Hough Transform** to detect lines in the point cloud:

$$r = x \cos\theta + y \sin\theta \tag{7}$$

where:

- r is the perpendicular distance from the origin to the line.
- θ is the angle of the normal vector to the line.

Each point (x, y) in the input space votes in a discretized accumulator space (r, θ).

Accumulator Space Peaks: Peaks in the accumulator space are identified, corresponding to candidate lines.

Line Parametrization: The detected lines are extracted and parametrized in terms of their endpoints:

$$\mathbf{l}_j = \{\mathbf{p}_s, \mathbf{p}_e\}, \quad \mathbf{p}_s, \mathbf{p}_e \in \mathbb{R}^2 \tag{8}$$

where $\mathbf{p}_s$ and $\mathbf{p}_e$ are the start and end points of the line.

4) Feature Decomposition: The method `RoomDetector::compute_features` returns a tuple containing:

- **ls**: A vector of line segments, each represented as a pair $(v, \texttt{QLineF})$, where v is the number of votes the line received in the Hough space, and `QLineF` is the Qt representation of the line.

5) Line Extraction: The algorithm iterates over the detected lines using the following loop:

```
for (const auto &l: ls)
    lines.emplace_back(l.second);
```

Here, `l.second` represents the `QLineF` object, which encapsulates the geometric definition of each line. The lines are stored in the output vector $\mathbf{L}$.

6) Visualization: The resulting lines are optionally drawn onto the provided graphical scene (`scene`) for verification and debugging.

7) Mathematical Details:

- **Hough Transform**: Each point (x_i, y_i) in the input space maps to a sinusoidal curve in the Hough space:

$$r_i(\theta) = x_i \cos\theta + y_i \sin\theta \tag{9}$$

  Peaks in the accumulator space correspond to lines passing through multiple points.
- **Thresholding**: Lines are accepted if their vote count v_j exceeds a predefined threshold v_{min}:

$$v_j \geq v_{\text{min}} \tag{10}$$

  where v_{min} ensures that only significant lines are detected.
- **Line Construction**: Each detected line in the Hough space is converted back into Cartesian coordinates by solving:

$$\mathbf{l}_j = \{(x_s, y_s), (x_e, y_e)\} \tag{11}$$

  where the endpoints are computed based on the intersection of the line with the bounding box of the environment.

8) Output Representation: The output is a set of lines $\mathbf{L} = \{\mathbf{l}_j\}$, where each line is represented in the `QLineF` format:

$$\mathbf{l}_j = \texttt{QLineF}(x_s, y_s, x_e, y_e) \tag{12}$$

C. Filtering of points close to the lines

This algorithm, implemented as `SpecificWorker::remove_wall_points`, filters out points from a given set that are located within a predefined distance from a set of walls. The walls are represented as line segments, and the algorithm evaluates the distance between each point and these lines to determine whether a point should be included in the output. Below is a detailed mathematical description of the algorithm.

1) Input:

- **Lines**: A set of wall segments, represented as $\mathbf{L} = \{\mathbf{l}_j : \mathbf{l}_j = (\mathbf{p}_1, \mathbf{p}_2)\}$, where each wall segment is defined by two endpoints $\mathbf{p}_1 = (x_1, y_1)$ and $\mathbf{p}_2 = (x_2, y_2)$ in $\mathbb{R}^2$.
- **Points**: A set of points $\mathbf{P} = \{\mathbf{p}_i \in \mathbb{R}^2\}$, representing the environment data such as LiDAR detections.
- **Threshold Distance**: A constant $d_{\text{threshold}} =$ `params.ROBOT_WIDTH`, representing the robot's width, which determines how far a point must be from a wall to be retained.

2) Output: A filtered set of points $\mathbf{P}_{\text{filtered}} = \{\mathbf{p}_i \in \mathbb{R}^2 : d(\mathbf{p}_i, \mathbf{l}_j) \geq d_{\text{threshold}}, \forall \mathbf{l}_j \in \mathbf{L}\}$, where each point is at least $d_{\text{threshold}}$ distance away from all walls.

3) Algorithm Steps: **Step 1: Iterate Over All Points**

For each point $\mathbf{p} \in \mathbf{P}$, initialize a flag $outside$ = true, assuming that the point is initially outside the threshold distance of all walls.

Step 2: Compute the Distance to Each Wall Segment

For each line $\mathbf{l}_j = (\mathbf{p}_1, \mathbf{p}_2) \in \mathbf{L}$, define a parametrized line ℓ_j passing through $\mathbf{p}_1$ and $\mathbf{p}_2$:

$$\ell_j(t) = \mathbf{p}_1 + t(\mathbf{p}_2 - \mathbf{p}_1), \quad t \in \mathbb{R}.$$

The shortest distance $d(\mathbf{p}, \mathbf{l}_j)$ from the point $\mathbf{p}$ to the line ℓ_j is computed as:

$$d(\mathbf{p}, \ell_j) = \frac{\|(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_1 - \mathbf{p})\|}{\|\mathbf{p}_2 - \mathbf{p}_1\|},$$

where $\|\cdot\|$ denotes the Euclidean norm, and $\times$ represents the 2D cross product.

Alternatively, this distance can be directly calculated using the Eigen library's `ParametrizedLine::distance()` method:

$$d(\mathbf{p}, \ell_j) = \texttt{pline.distance(p)}.$$

Step 3: Check Distance Threshold

If $d(\mathbf{p}, \ell_j) < d_{\text{threshold}}$, set $outside$ = false and stop checking further lines for the current point. This indicates that the point is within the threshold distance of a wall and should not be included in the output.

Step 4: Retain or Discard the Point

If $outside$ remains true after all lines have been checked, append $\mathbf{p}$ to the output set $\mathbf{P}_{\text{filtered}}$.

4) Mathematical Representation: The filtered set of points $\mathbf{P}_{\text{filtered}}$ is computed as:

$$\mathbf{P}_{\text{filtered}} = \left\{ \mathbf{p} \in \mathbf{P} : \min_{\mathbf{l}_j \in \mathbf{L}} d(\mathbf{p}, \mathbf{l}_j) \geq d_{\text{threshold}} \right\}.$$

5) Output Representation: The algorithm returns $\mathbf{P}_{\text{filtered}}$, a set of points in $\mathbb{R}^2$ that are at least $d_{\text{threshold}}$ distance away from all lines in $\mathbf{L}$.

D. Computation of polygons using DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a popular clustering algorithm that identifies clusters of points based on their spatial density. It is particularly useful in scenarios where the clusters are not necessarily spherical and can have arbitrary shapes. The algorithm operates using two key parameters:

- **eps** (epsilon): This parameter defines the maximum distance between two points for them to be considered neighbors.
- **min_points**: The minimum number of points required to form a dense region (i.e., a valid cluster).

Mathematically, DBSCAN works by categorizing points into three types:

- **Core points**: A point is considered a core point if there are at least `min_points` points within a radius of `eps` from it. These points serve as the starting points for a cluster.
- **Border points**: A point that is within the `eps` radius of a core point but does not have enough points around it to be considered a core point.
- **Noise points**: A point that is neither a core point nor a border point. These points do not belong to any cluster.

The algorithm proceeds by:

- Starting from an arbitrary unvisited point, it looks for neighboring points within the `eps` distance. If the number of neighbors is greater than or equal to `min_points`, a new cluster is initiated.
- The process is repeated for all reachable points, recursively expanding the cluster by adding neighboring points that are also core points.
- Points that cannot be assigned to a cluster (because they do not meet the density criteria) are labeled as noise.

The DBSCAN algorithm's ability to detect clusters of arbitrary shapes and handle noise makes it ideal for detecting groups of points in 2D spaces, such as those generated by a robot's lidar sensor.

Given a set of points $P = \{p_1, p_2, \ldots, p_n\}$ in a 2D space, DBSCAN works as follows:

- **Neighborhood**: The neighborhood of a point p_i is defined as the set of points $N(p_i)$ that lie within a distance ϵ from p_i:

  $$N(p_i) = \{p_j \in P \mid \text{distance}(p_i, p_j) \leq \epsilon\}$$

  where $\text{distance}(p_i, p_j)$ is typically the Euclidean distance:

  $$\text{distance}(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

- **Core Point**: A point p_i is a core point if $|N(p_i)| \geq$ min_points, where $|N(p_i)|$ denotes the number of points within the neighborhood $N(p_i)$.
- **Cluster Formation**: A new cluster is formed by iteratively expanding from a core point, adding all reachable points within ϵ, and then continuing this process for newly discovered core points.
- **Convex Hull Computation**: After clustering, the convex hull of each cluster is computed. The convex hull is the smallest convex polygon that contains all points of a given cluster. The convex hull of a set of points $S = \{p_1, p_2, \ldots, p_m\}$ is defined as the minimal convex set that contains S.

The convex hull can be computed using algorithms such as Graham Scan or QuickHull. The convex hull of a set of points is the boundary that encloses all points, forming a polygon.

ENLARGING POLYGONS ALGORITHM

The `enlarge_polygons` function is designed to expand polygons by a specified amount. This process involves iterating over each polygon and modifying its vertices to create a larger version of the polygon. The function works as follows:

- The function accepts a vector of polygons and a float `amount` that specifies how much to enlarge the polygons.
- For each polygon in the input vector:

  - A copy of the polygon is made, and the first point is appended to the end of the polygon to ensure the loop can operate cyclically on the points.
  - For each consecutive triplet of points (using a sliding window of size 3), the algorithm computes a bisectrix vector that represents the direction of expansion between the points.
  - The polygon is enlarged by moving each point outward in the direction of the bisectrix, scaled by the specified `amount`.
- The result is a new set of enlarged polygons, each of which is the original polygon expanded outward by the given amount.

The function returns a vector of expanded polygons. The algorithm ensures that polygons with less than three points are skipped, as they are not suitable for this type of expansion.

Mathematical Representation

Let the polygon be represented by a sequence of vertices $P = \{p_1, p_2, \ldots, p_n\}$, where each point $p_i = (x_i, y_i)$ in the polygon is a 2D point. The function works by iterating over the vertices and expanding the polygon as follows:

- For each consecutive triplet of points (p_1, p_2, p_3), the bisectrix is calculated by normalizing the direction vectors between the points:

$$\text{bisectrix} = \frac{(p_1 - p_2)}{\|p_1 - p_2\|} + \frac{(p_3 - p_2)}{\|p_3 - p_2\|}.$$

  The result is then normalized again:

$$\text{bisectrix} = \frac{\text{bisectrix}}{\|\text{bisectrix}\|}.$$

- The new point p_2' (expanded point) is calculated by moving p_2 outward along the bisectrix by an amount a:

$$p_2' = p_2 + a \times \text{bisectrix}.$$

This operation is performed for all points in the polygon, resulting in the enlargement of the entire polygon.

The function ensures that polygons with fewer than three points are skipped, as they do not form a valid shape to be expanded.

E. Computation of line polygons

Generating Wall Polygons Algorithm

The `get_walls_as_polygons` function converts a set of lines (representing walls) into polygons that represent the boundaries of these walls, with a width corresponding to the robot's size. The function works as follows:

- The function accepts a vector of lines and a float `robot_width`, which specifies the width of the robot.
- For each line in the input vector:
  - The direction vector of the line is computed by subtracting the coordinates of the starting point of the line from the endpoint.
  - The normal vector to the line is calculated by rotating the direction vector by 90 degrees.
  - The normal vector is then normalized to unit length.
  - Two new points are created along the normal vector at both ends of the line, using the robot's width to determine the offset. This results in a quadrilateral that represents the wall.
- The function then creates a polygon from the four points and adds it to the list of obstacles.
- The result is a vector of polygons, each representing the walls with the robot's width taken into account.

The function essentially "thickens" each line to represent the actual physical space occupied by the walls relative to the robot.

Mathematical Representation

Let the wall be represented by a line segment $L = (p_1, p_2)$, where $p_1 = (x_1, y_1)$ and $p_2 = (x_2, y_2)$ are the start and end points of the line.

The algorithm computes the following:

- The direction vector of the line:

$$\vec{d} = (x_2 - x_1, y_2 - y_1).$$

- The normal vector $\vec{n}$, which is perpendicular to the direction vector:

$$\vec{n} = (-\Delta y, \Delta x),$$

$$\text{where} \quad \Delta x = x_2 - x_1 \quad \text{and} \quad \Delta y = y_2 - y_1.$$

- The normal vector is then normalized:

$$\hat{n} = \frac{\vec{n}}{\|\vec{n}\|}.$$

- The new points are computed by displacing the start and end points along the normal vector by a distance of $\frac{\text{robot_width}}{2}$. Thus, the four corners of the polygon are given by:

$$\text{Point 1} = p_1 + \frac{\text{robot_width}}{2} \cdot \hat{n}, \quad \text{Point 2} = p_2 + \frac{\text{robot_width}}{2} \cdot \hat{n},$$

$$\text{Point 3} = p_2 - \frac{\text{robot_width}}{2} \cdot \hat{n}, \quad \text{Point 4} = p_1 - \frac{\text{robot_width}}{2} \cdot \hat{n}.$$

These four points are then used to form a quadrilateral, which is returned as a polygon representing the wall. This process is repeated for each line in the input.

The result is a set of polygons, each representing the boundaries of the walls, with a width equal to the robot's size, ensuring accurate collision detection and navigation around obstacles.

After the computation, we have to add the result obtained to the group of polygons so we can compute this changes in the execution of the simulation.

F. Delaunay triangularisation and Voronoi diagram

Delaunay triangulation is a geometric algorithm that divides a set of 2D points into a network of triangles, ensuring that no point lies inside the circumcircle of any triangle. This property maximizes the minimum angle of the triangles, avoiding elongated shapes and providing a uniform division of the free space. In this project, Delaunay triangulation was used to process the LiDAR data, specifically the filtered points representing the robot's navigable free space.

a) Input Points: The triangulation starts with a set of 2D points $P = \{p_1, p_2, \ldots, p_n\}$, representing LiDAR-detected free-space points. Points associated with obstacles or walls, identified using the DBSCAN clustering algorithm are excluded.

b) Triangulation Process: Each triplet of points (p_i, p_j, p_k) is evaluated to check if its circumcircle, defined by:

$$(x - x_c)^2 + (y - y_c)^2 = r^2,$$

where (x_c, y_c) is the circle's center and r its radius, contains any other point from P. Triangles satisfying this criterion are included in the Delaunay triangulation, forming a connected graph of non-overlapping triangles.

c) Voronoi Diagram Construction: The Voronoi diagram is derived as the dual of the Delaunay triangulation. For each point $p_i \in P$, a Voronoi cell V_i is defined as:

$$V_i = \{x \in \mathbb{R}^2 : \|x - p_i\| \leq \|x - p_j\|, \forall p_j \neq p_i\}.$$

Voronoi edges connect points equidistant from two neighboring nodes, ensuring paths are maximally separated from obstacles.

d) Graph Construction for Navigation: The centroids of the Delaunay triangles are used as nodes in a visibility graph, and the edges represent potential paths. This graph ensures a robust representation of free space for navigation.

The Delaunay triangulation and the corresponding Voronoi diagram provide a detailed map of the robot's environment, enabling the division of free space into manageable regions and supporting safe path planning.

G. Calculation of optimal path in Voronoi diagram using Djikstra algorithm

With the visibility graph constructed from the Voronoi diagram, Dijkstra's algorithm is used to find the shortest path between the robot's current position and Eva. The process is detailed below:

a) Graph Representation: Let $G = (V, E)$ represent the visibility graph, where V is the set of centroids of the Delaunay triangles, and E are the edges connecting centroids

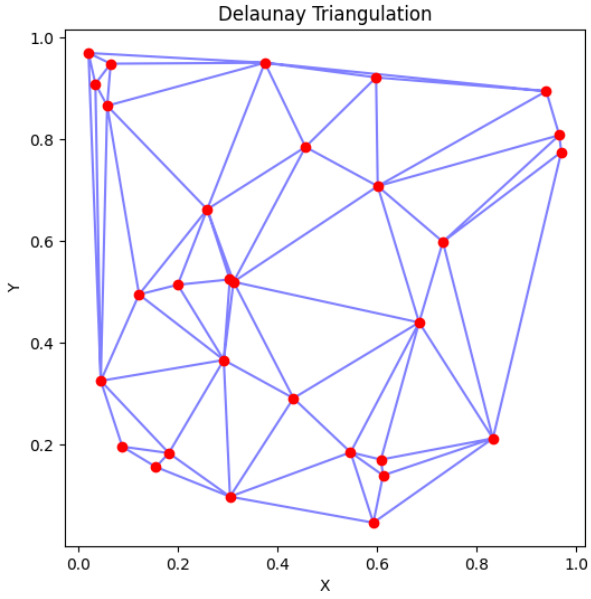


Fig. 13. (a) Delaunay triangulation of free-space points, and (b) Voronoi diagram constructed as its dual.

of adjacent triangles. The weight of each edge $e_{ij} \in E$ is calculated as the Euclidean distance between nodes v_i and v_j:

$$w(e_{ij}) = \|v_i - v_j\| = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

b) Initialization: The robot's position serves as the starting node s, and Eva's detected location, determined using YOLO and LiDAR data, is assigned to the nearest centroid as the goal node g. All nodes are initialized with a tentative distance of infinity, except $d[s] = 0$.

c) Relaxation and Pathfinding: The algorithm iterates over the nodes, updating the shortest known distance to its neighbors:

$$d[v] = \min(d[v], d[u] + w(e_{uv})).$$

d) Path Reconstruction: Once the shortest path to g is identified, it is reconstructed by tracing back from g to s using the recorded predecessors.

e) Path Smoothing and Discretization: The resulting path is smoothed using cubic splines:

$$x(t) = a_x t^3 + b_x t^2 + c_x t + d_x, \quad y(t) = a_y t^3 + b_y t^2 + c_y t + d_y,$$

ensuring smooth transitions between waypoints. The smoothed path is discretized into smaller steps to be sent to the robot's proportional-derivative (PD) controller for execution.

f) Dynamic Updates: If Eva moves or if new obstacles are detected, the system recalculates the visibility graph and updates the optimal path in real time, ensuring the robot adapts dynamically to changes in the environment.

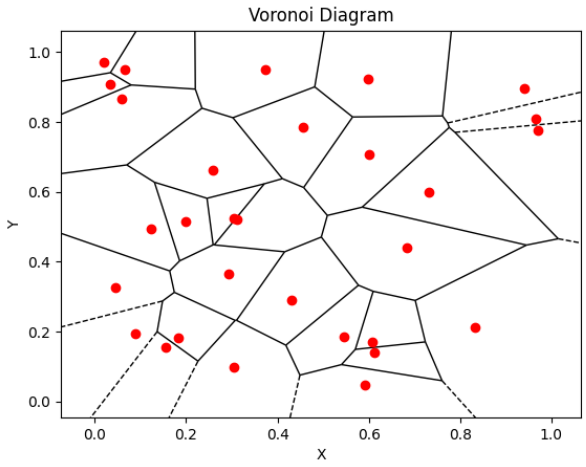


Fig. 14. (a) Visibility graph derived from the Voronoi diagram, (b) shortest path using Dijkstra's algorithm, and (c) smoothed trajectory for execution.

By leveraging the Voronoi diagram and Dijkstra's algorithm, the robot efficiently navigates through free space, avoiding obstacles while following Eva.

H. Path Following using the state machine logic

This algorithm, implemented as `SpecificWorker::state_machine`, enables a robot to follow a given path while dynamically switching between predefined behavioral states: `TRACK`, `WAIT`, `SEARCH`, and `STOP`. The transitions between states are based on sensory input and the robot's interaction with the environment. Below is a detailed mathematical and procedural description of the algorithm.

1) Input:

- **Path:** A sequence of 2D points $\mathbf{P} = \{\mathbf{p}_i \in \mathbb{R}^2 : \mathbf{p}_i = (x_i, y_i)\}$, representing the robot's desired trajectory towards the target.
- **State:** The current operational state of the robot, defined as $S \in \{\texttt{TRACK}, \texttt{WAIT}, \texttt{SEARCH}, \texttt{STOP}\}$.
- **Parameters:**
  - k_1, k_2: Gains for proportional-derivative (PD) control of angular error.
  - `MAX_ADV_SPEED`: Maximum forward speed of the robot.
  - `ROBOT_WIDTH`: Physical width of the robot.
  - `PERSON_MIN_DIST`: Minimum allowed distance to the target.
  - `SEARCH_ROT_SPEED`: Angular velocity used during the search state.

2) Output: The algorithm produces the following outputs:

- **Speed:** Linear velocity $v \in \mathbb{R}$ of the robot.
- **Rotation:** Angular velocity $\omega \in \mathbb{R}$ of the robot.

3) State Machine Logic: The state machine determines the robot's behavior by selecting an appropriate state-specific function. The logic is expressed as:

$$(S', v, \omega) = f(S, \mathbf{P}),$$

where S' is the next state, and f depends on the current state S and the path $\mathbf{P}$. The following transitions occur:

- **`TRACK`:** Actively follows the path using PD control. If the target distance is below `PERSON_MIN_DIST`, the state transitions to `WAIT`.
- **`WAIT`:** Stops the robot if the target is close and transitions to `TRACK` when the target moves beyond a threshold.
- **`SEARCH`:** Rotates the robot in place to locate a target and transitions to `TRACK` upon detecting the target.
- **`STOP`:** Halts the robot completely until manually resumed, transitioning to `TRACK` otherwise.

4) Control Laws: **1. Proportional-Derivative (PD) Angular Control:**

The angular error θ_{error} between the robot's orientation and the next path point (x_i, y_i) is computed as:

$$\theta_{\text{error}} = \arctan 2(y_i, x_i),$$

where $\arctan 2$ accounts for the quadrant of the target point. The rotational velocity ω is determined by the PD controller:

$$\omega = k_1 \cdot \theta_{\text{error}} + k_2 \cdot (\theta_{\text{error}} - \theta_{\text{error}}^{\text{prev}}),$$

where $\theta_{\text{error}}^{\text{prev}}$ is the previous angular error.

2. Speed Modulation Based on Angular and Linear Braking:

The forward speed v is modulated based on:

- **Rotational Braking:** A Gaussian function $g(\omega)$ reduces speed for large angular errors:

$$g(\omega) = \exp\left(-\frac{\omega^2}{s}\right),$$

  where s is the variance derived from user-defined parameters.
- **Distance-Based Braking:** Linear braking is applied as:

$$\text{adv_brake} = \text{clamp}\left(\frac{d}{\text{acc_distance}} - \frac{\text{PERSON_MIN_DIST}}{\text{acc_distance}}\right),$$

  where d is the distance to the target along the path, and acc_distance is a parameter defining the deceleration zone.

The final forward speed is:

$$v = \texttt{MAX_ADV_SPEED} \cdot g(\omega) \cdot \texttt{adv_brake}.$$

5) State-Specific Behavior: **1. `TRACK`:** This state uses the control laws described above to align the robot with the target and move forward. If the target distance d falls below `PERSON_MIN_DIST`, the state transitions to `WAIT`.

2. `WAIT`: The robot stops movement:

$$v = 0, \quad \omega = 0.$$

If the target distance d exceeds `PERSON_MIN_DIST` + threshold, the state transitions back to `TRACK`.

3. `SEARCH`: In this state, the robot rotates in place to locate the target:

$$v = 0, \quad \omega = \texttt{SEARCH_ROT_SPEED}.$$

If a target is found, the state transitions to `TRACK`.

4. `STOP`: The robot halts completely:

$$v = 0, \quad \omega = 0.$$

The state persists until manually overridden.

6) Output Representation: The robot's final speed (v, ω) is determined by the selected state and control laws. The output is:

$$\text{Robot Speed} = (v, \omega).$$

I. Execution example

Here is the **link** to the video where we can see the simulation of the code.

VI. Activity 4: Avoiding Obstacles While Following Eva

A. Problem Description

The primary objective of this activity is to replace the reactive obstacle-avoidance strategy implemented in the previous activity with a robust, deterministic approach capable of handling dynamic and static obstacles more efficiently. The proposed solution leverages a **2D grid representation** of the robot's surrounding environment to enable advanced path-planning algorithms such as Dijkstra. This design addresses the limitations observed in the earlier approach, particularly in scenarios with inconsistent sensor data and suboptimal path-following behavior.

The grid-based representation models the robot's environment as discrete cells, where each cell represents a $100\,\text{mm}$ x $100\,\text{mm}$ square in the real world. By dynamically updating the grid state (free, occupied, or unknown) based on LiDAR sensor readings, the robot gains a more accurate and scalable understanding of its surroundings. The introduction of weighted cells allows for prioritizing safer or more optimal paths during planning.

Additionally, the use of a **dedicated Grid2D component** enables seamless interaction between LiDAR sensors and the path-planning system. This modular design reduces computation latency by enabling multi-core parallelization for grid updates and path calculations.

In the earlier approach, reactive mechanisms for obstacle avoidance often led to unpredictable behavior, especially in cluttered environments. This was attributed to the lack of a global representation of the environment and the reliance on immediate sensor feedback alone. Such behavior can lead to suboptimal paths, oscillations near obstacles, and difficulty in navigating toward the target efficiently.

The proposed grid-based method addresses these shortcomings by:

- **Providing a stable representation:** The grid encapsulates the robot's local environment, ensuring a consistent frame of reference even under noisy sensor conditions.
- **Enabling deterministic path planning:** The application of Dijkstra's algorithm ensures that the robot can compute optimal and collision-free paths to its target while considering known obstacles.
- **Supporting modular scalability:** By offloading path-planning responsibilities to a dedicated component, the system architecture becomes more maintainable and adaptable to future extensions.

This solution marks a significant advancement in ensuring the robot's ability to follow a human target (e.g., Eva) in dynamic indoor scenarios, such as a home or kitchen, where obstacle density and unpredictability are common.

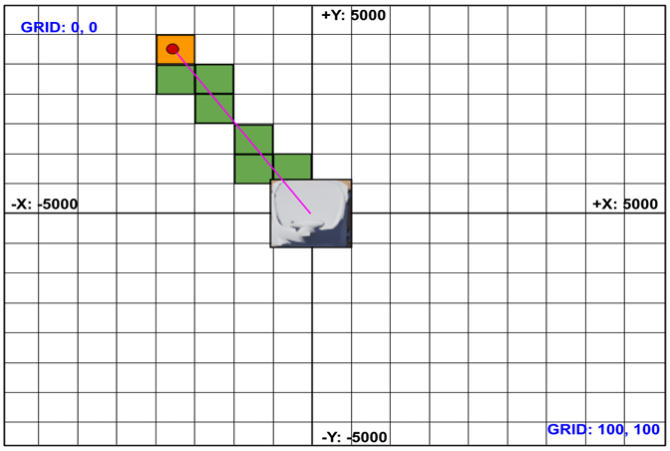


Fig. 15. Representation of the path to the target in the grid

B. Decomposition in Two Components

The proposed solution is divided into two main components: the **Obstacle and Person Tracker (ObsPersonTracker)** and the **Grid2D Planner**. This decomposition ensures modularity, scalability, and efficient collaboration between different functionalities required for the robot's operation.

1) ObsPersonTracker Component: The **ObsPersonTracker** component is responsible for detecting and tracking the target (e.g., a person) in the environment. It integrates the YOLO object detection system to identify the person's position in real-time. This component also collects and preprocesses data from the LiDAR sensors (e.g., BPEARL and Helios) to identify potential obstacles in the robot's immediate vicinity.

The core responsibilities of this component include:

- Identifying the target's position and orientation using YOLO object detection and LiDAR data.

- Relaying the target's coordinates to the **Grid2D Planner**.
- Handling real-time adjustments when the target moves unpredictably or when obstacles are dynamically introduced into the scene.
- Filtering and drawing LiDAR data to visualize obstacles for debugging and monitoring purposes.

2) Grid2D Planner Component: The **Grid2D Planner** component is responsible for maintaining a 2D grid representation of the environment and planning an optimal, obstacle-free path to the target. This component models the environment as a grid of cells, each representing a $100\,\mathrm{mm}$ x $100\,\mathrm{mm}$ square. The cells can have one of three states: *free*, *occupied*, or *unknown*. The grid is updated dynamically based on LiDAR readings.

The key functionalities of this component include:

- Dynamically updating the grid based on LiDAR data by marking cells as *free* or *occupied*.
- Employing Dijkstra's algorithm to compute the optimal path from the robot's current position (grid center) to the target's position.
- Sending the computed path to the **ObsPersonTracker** for execution.
- Handling concurrency issues by synchronizing grid updates and path planning.

3) Communication Between Components: The communication between the two components is achieved using the **Grid2D interface**. This interface, defined in the `Grid2D.idsl` file, facilitates seamless interaction between the **ObsPersonTracker** and **Grid2D Planner** components. The interface defines the following key method:

- `Result getPaths(TPoint source, TPoint target)`: This method allows the **ObsPersonTracker** to request an obstacle-free path from the **Grid2D Planner** by providing the source (robot position) and target (person's position) coordinates.

The **Grid2D Planner** computes the path and returns it as a sequence of waypoints, along with metadata such as timestamps and error messages (if applicable). The **ObsPersonTracker** then processes this path to command the robot's actuators, enabling it to navigate effectively.

This modular design ensures that both components operate independently, yet remain tightly coupled through a well-defined communication interface. This approach enhances reusability and maintainability while allowing for the parallelization of tasks such as grid updates and path execution.

C. The `Grid2D.idsl` File and Component Integration

The `Grid2D.idsl` file defines the communication interface used in the system to enable seamless interaction between the **ObsPersonTracker** and **Grid2D Planner** components. Additionally, the `grid2D.cdsl` file specifies how the components interact and integrate into the broader system architecture, ensuring modularity and scalability.

1) `Grid2D.idsl`: Interface Definition: The `Grid2D.idsl` file provides the structural foundation for the path-planning functionalities by defining key data types and methods. These include:

- **Data Structures:**
  - `TPoint`, which represents a 2D point in the environment, including its coordinates and bounding radius.
  - `TPath`, a sequence of 2D points that defines the path the robot must follow.
  - `Result`, a structure that encapsulates the computed path, metadata such as timestamps, and error information.
- **Core Method:**
  - The interface defines a single method that computes a path between two points (source and target) while avoiding obstacles.

This interface ensures clear separation of responsibilities, allowing the path-planning logic to operate independently of the tracking and sensor systems.

2) `grid2D.cdsl`: Component Integration: The `grid2D.cdsl` file is responsible for specifying the integration of components and their dependencies within the system. It defines the `grid2D` component, which:

- Implements the `Grid2D` interface, providing the path-planning functionalities defined in `Grid2D.idsl`.
- Requires the `Lidar3D` interface to process LiDAR data for grid updates and obstacle detection.
- Specifies that the component is implemented in C++ and includes a graphical user interface for visualization and debugging purposes.

3) Importance in the System Architecture: The `Grid2D.idsl` and `grid2D.cdsl` files play complementary roles in the system. The `Grid2D.idsl` file ensures that the interface is well-defined, enabling efficient communication between the **ObsPersonTracker** and **Grid2D Planner**. Meanwhile, the `grid2D.cdsl` file facilitates the integration of these components, ensuring that the system operates as a cohesive unit.

By adopting this modular design, the system achieves several benefits:

- **Reusability:** Components can be independently reused or replaced without affecting the rest of the system.
- **Scalability:** New functionalities or interfaces can be easily added without significant architectural changes.
- **Maintainability:** Decoupling components simplifies debugging, testing, and future development.

This combination of well-defined interfaces and modular integration ensures the robustness and flexibility of the robotic system, enabling it to perform reliable path-planning and obstacle avoidance in dynamic environments.

D. Grid Representation for Maintaining the Free/Occupied State of the World

The grid-based representation is a central component of the system, providing a spatial model of the robot's environment.

This representation allows for efficient obstacle detection, path planning, and navigation. The environment is discretized into a 2D grid where each cell corresponds to a square area of $100\,\text{mm}$ x $100\,\text{mm}$. The state of each cell is dynamically updated based on sensor data, and mathematical operations are used to map real-world coordinates onto the grid and vice versa.

1) Grid Design and Initialization: The grid is defined as a fixed-size 2D array:

$$\text{Grid} \in \mathbb{R}^{n \times m},$$

where n and m represent the number of cells along the X and Y axes, respectively. In this implementation, the grid size corresponds to a world of $10\,\text{m}$ x $10\,\text{m}$, resulting in $n = m = 100$ cells.

Each cell in the grid, denoted as $c_{i,j}$, stores the following attributes:

- **State (`state`):** The occupancy state of the cell, which can be:
  - `Free`: No obstacle is present.
  - `Occupied`: An obstacle is present.
  - `Unknown`: The state is undetermined.
- **Visualization Element (`item`):** A graphical object used for visualizing the cell in the user interface.
- **Changed Flag (`changed`):** Indicates whether the cell's state has been updated in the current cycle.

2) Coordinate Transformation: The grid is centered on the robot, with the robot's position at $(0, 0)$ in the real world corresponding to the center of the grid. To map real-world coordinates (x, y) to grid indices (i, j), the following transformation is applied:

$$i = \left\lfloor \frac{x + W/2}{\text{Cell Size}} \right\rfloor, \quad j = \left\lfloor \frac{y + H/2}{\text{Cell Size}} \right\rfloor,$$

where:

- W and H are the width and height of the grid in millimeters.
- Cell Size is the size of each cell ($100\,\text{mm}$).

The inverse transformation, which maps grid indices (i, j) back to real-world coordinates, is given by:

$$x = i \cdot \text{Cell Size} - W/2, \quad y = j \cdot \text{Cell Size} - H/2.$$

3) Updating the Grid State: At each iteration, the grid is updated using data from the LiDAR sensors. The update process involves:

1) Initializing the entire grid to the `Free` state.
2) Processing each LiDAR point (x_p, y_p) to determine the cells it intersects. This is done by tracing a parametric line from the robot's position to the LiDAR point:

$$\mathbf{R}(t) = t\mathbf{P}, \quad 0 \leq t \leq 1,$$

where:

- $\mathbf{P} = (x_p, y_p)$ is the endpoint of the LiDAR ray.
- t is the interpolation parameter, with steps of $\Delta t = \frac{\text{Cell Size}}{\|\mathbf{P}\|}$.

3) Marking the traversed cells as `Free`, except for the last cell, which is marked as `Occupied`.

4) Mathematical Considerations for Collision Expansion: To ensure safe navigation, an additional step is performed to expand the boundaries of detected obstacles. Each `Occupied` cell propagates its state to its neighboring cells within a margin of k cells, defined as:

$$k = \left\lceil \frac{\text{Safety Margin}}{\text{Cell Size}} \right\rceil.$$

This ensures that the robot avoids regions near obstacles, accounting for potential inaccuracies in sensor data or control delays.

5) Visualization and Debugging: Each cell in the grid is associated with a graphical object that represents its state:

- `Free` cells are drawn in white.
- `Occupied` cells are drawn in red.
- `Unknown` cells are drawn in light gray.

This visualization allows developers to monitor the system's performance and verify the correctness of the grid updates.

6) Advantages of the Grid Representation: The grid representation provides several benefits:

- **Simplicity:** Discretizing the environment simplifies path-planning algorithms such as Dijkstra.
- **Scalability:** The fixed-size array enables efficient updates and supports real-time operations.
- **Flexibility:** The modular design allows integration with other components, such as obstacle detection and tracking systems.

By using this representation, the system can accurately model its surroundings, enabling reliable and efficient navigation in dynamic environments.

E. Core Methods of the `Gridder` Component (Update and Plan)

The `Gridder` component is responsible for maintaining the grid representation of the environment and computing optimal paths to targets. Its two core methods, **update** and **plan**, form the backbone of the system's ability to adapt to dynamic environments and navigate efficiently.

1) `update_grid`: Maintaining the Grid State: The `update_grid` method processes data from LiDAR sensors to dynamically update the occupancy state of the grid cells. The method performs the following steps:

1. Transforming LiDAR Points to Grid Coordinates:

Each point (x_p, y_p) from the LiDAR is mapped to grid indices (i, j) using the transformation:

$$i = \left\lfloor \frac{x_p + W/2}{\text{Cell Size}} \right\rfloor, \quad j = \left\lfloor \frac{y_p + H/2}{\text{Cell Size}} \right\rfloor,$$

where W and H represent the grid dimensions in millimeters, and $\text{Cell Size} = 100\,\text{mm}$.

2. Marking Free and Occupied Cells:

For each LiDAR point, a parametric line is traced from the robot's position $(0, 0)$ to the LiDAR endpoint (x_p, y_p):

$$\mathbf{R}(t) = t\mathbf{P}, \quad 0 \leq t \leq 1,$$

where:

- $\mathbf{P} = (x_p, y_p)$ is the LiDAR point.
- t is incremented in steps of $\Delta t = \frac{\text{Cell Size}}{\|\mathbf{P}\|}$.

As the line is traversed:

- The intersected cells are marked as `Free`.
- The final cell corresponding to (x_p, y_p) is marked as `Occupied`.

3. Obstacle Expansion:

To account for the robot's physical dimensions and control delays, the `regruesadoObstaculo` method expands each `Occupied` cell into a surrounding margin:

$$k = \left\lceil \frac{\text{Safety Margin}}{\text{Cell Size}} \right\rceil,$$

where k is the number of cells expanded in each direction. Neighboring cells within this margin are also marked as `Occupied`.

2) `dijkstraAlgorithm`: Path Planning: The `dijkstraAlgorithm` method computes an optimal path from the robot's current position to a specified target while avoiding obstacles. The algorithm uses the following steps:

1. Graph Representation: The grid is treated as a graph, where:

- Each cell (i, j) represents a node.
- Edges connect neighboring nodes (4 or 8 neighbors, depending on connectivity).
- Edge weights are uniform for `Free` cells and infinite for `Occupied` cells.

2. Dijkstra's Algorithm

The algorithm minimizes the cost function:

$$C(u, v) = C(u) + w(u, v),$$

where:

- u and v are neighboring nodes.
- $C(u)$ is the cumulative cost to reach node u.
- $w(u, v)$ is the weight of the edge between u and v.

The steps are:

1) Initialize a priority queue with the start node, setting its cost to 0 and all others to infinity.
2) For the current node, update the costs of its neighbors if a lower-cost path is found.
3) Mark the current node as visited and repeat until the target is reached or the queue is empty.

3. Path Reconstruction

Once the target is reached, the path is reconstructed by backtracking through the predecessor nodes:

$$\text{Path} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\},$$

where (x_1, y_1) is the start and (x_n, y_n) is the target.

3) Mathematical Considerations:

Cell Traversal: The traversal of cells is governed by the parametric equation of a line. Given the resolution of the grid, the number of steps is calculated as:

$$S = \frac{\|\mathbf{P}\|}{\text{Cell Size}},$$

ensuring that each cell along the path is accurately updated.

Grid Connectivity: The choice of connectivity (4 or 8 neighbors) impacts the path quality. Using 8-connectivity allows diagonal movement, reducing the overall path length:

$$\text{Path Length} = \sum_{i=1}^{n-1} \|\mathbf{P}_i - \mathbf{P}_{i+1}\|,$$

where $\mathbf{P}_i$ are consecutive waypoints.

4) Advantages of the Core Methods: The combination of `update_grid` and `dijkstraAlgorithm` provides:

- **Real-time adaptability:** The grid is updated dynamically based on sensor input.
- **Optimal navigation:** The use of Dijkstra ensures collision-free, efficient paths.
- **Scalability:** These methods can be extended to larger grids or more complex environments.

Together, these methods enable the robot to navigate complex, dynamic environments reliably and efficiently.

F. Concurrency Issues in Simultaneous Grid Updates and Path Requests

Concurrency is a critical aspect of the `Gridder` component, as it handles simultaneous operations such as grid updates based on sensor data and path-planning requests from the `ObsPersonTracker`. Without appropriate synchronization, these operations can lead to race conditions, inconsistent states, and incorrect navigation behaviors.

1) Overview of Concurrency Challenges: The two primary tasks that can run concurrently are:

- **Grid updates:** The `update_grid` method processes incoming LiDAR data to update the state of the grid cells. This involves marking cells as `Free`, `Occupied`, or expanding obstacles to account for safety margins.
- **Path-planning requests:** The `dijkstraAlgorithm` method computes the shortest path from the robot's current position to a target, relying on the current grid state to determine free and occupied cells.

If these tasks execute simultaneously without proper control, the following issues may arise:

1) **Inconsistent grid states:** If the grid is updated while a path-planning algorithm is reading its state, the algorithm may encounter partially updated or incorrect cell states.
2) **Race conditions:** Multiple threads accessing and modifying the same grid data simultaneously may result in unexpected behavior, such as invalid memory access or incomplete obstacle expansion.
3) **Deadlocks:** Poorly implemented synchronization mechanisms can lead to situations where both tasks block each other indefinitely.

2) Mathematical Analysis of Concurrency Issues: Let $G \in \mathbb{R}^{n \times m}$ represent the grid, where $c_{i,j} \in G$ denotes the state of a cell. At any given time, the state $c_{i,j}$ can transition between `Free`, `Occupied`, and `Unknown`.

Grid State Consistency:

The path-planning algorithm $P(t)$ depends on the grid state $G(t)$. If $G(t)$ is being modified during the execution of $P(t)$, the algorithm may use partially updated values:

$$P(t) = f(G(t)), \quad G(t) = \{c_{i,j}(t)\}.$$

If $c_{i,j}(t)$ transitions mid-computation, the function f (Dijkstra's algorithm) may produce an incorrect or incomplete path, as its input is no longer consistent.

Race Conditions in Cell Access:

Assume two threads:

- T_1: Updates $c_{i,j}$ during the grid update process.
- T_2: Reads $c_{i,j}$ during path planning.

If both threads access $c_{i,j}$ simultaneously without synchronization, the resulting state $c'_{i,j}$ is undefined:

$$c'_{i,j} = \text{interleaved}(c_{i,j}^{T_1}, c_{i,j}^{T_2}),$$

where interleaved represents a non-deterministic combination of operations from T_1 and T_2.

Deadlock Analysis:

Synchronization mechanisms, such as mutexes, are often used to avoid concurrent access. Let μ represent a mutex protecting G. A deadlock occurs if:

$$T_1 \text{ locks } \mu \text{ and waits for } T_2, \quad T_2 \text{ locks } \mu \text{ and waits for } T_1.$$

The condition for deadlock can be expressed using the following inequalities:

$$\mu_{T_1} + \mu_{T_2} = 2, \quad \text{(both threads hold the mutex simultaneously).}$$

3) Mitigation Strategies: To address these concurrency issues, the following strategies are implemented:

1. Mutex Locking:

A mutex (`std::mutex`) is used to protect the grid structure G. Before either task accesses the grid, it must acquire the lock:

$$\text{lock}(\mu), \quad \text{access } G, \quad \text{unlock}(\mu).$$

This ensures that only one thread can modify or read the grid at a time, preventing race conditions.

2. Read-Write Locks:

To optimize performance, a read-write lock (`std::shared_mutex`) can be used:

- Multiple threads can read G simultaneously.
- Only one thread can write G, and it blocks all readers.

This allows the path-planning algorithm to read the grid state while updates are not being performed.

3. Atomic Updates:

For cells that are frequently updated, atomic operations (`std::atomic`) ensure that individual updates to $c_{i,j}$ are thread-safe without requiring a global lock.

4. Double Buffering:

A double-buffering approach is employed to separate reading and writing operations:

$$G_{\text{current}} : \text{Used by path planning}, \quad G_{\text{next}} : \text{Updated by grid updates.}$$

Once updates are complete, the buffers are swapped atomically:

$$G_{\text{current}}, G_{\text{next}} \leftarrow G_{\text{next}}, G_{\text{current}}.$$

4) Impact on System Performance: The use of these concurrency control mechanisms ensures:

- **Data Integrity:** The grid state remains consistent during concurrent operations.
- **Real-Time Capability:** Tasks can execute with minimal delays, maintaining responsiveness.
- **Scalability:** The system can handle more complex environments and increased sensor data without compromising reliability.

By carefully managing concurrency, the system avoids pitfalls such as race conditions, inconsistent states, and deadlocks, ensuring reliable performance in dynamic environments.

G. Interaction Between the Obs Component (Tracker) and the Gridder for Path Planning

The `ObsPersonTracker` component, also referred to as the `Tracker`, is responsible for detecting and tracking the target's position (e.g., a person) in real-time. This component interacts with the `Gridder` to compute an optimal path to the target based on the grid representation of the environment. The interaction is governed by specific triggers and conditions within the system's operational cycle.

1) When the Tracker Calls the Gridder: The `Tracker` initiates a call to the `Gridder` component under the following conditions:

- **Target Detection:** The tracker continuously processes data from YOLO object detection and LiDAR sensors. When a target is detected with sufficient confidence (`score > 0.6`), the tracker determines the target's coordinates (x_t, y_t).
- **Path Recalculation Trigger:** A new path is requested from the `Gridder` when:
  - The target position changes significantly, defined as:
    $$\|\mathbf{T}_{\text{new}} - \mathbf{T}_{\text{current}}\| > \Delta d,$$
    where Δd is a predefined distance threshold.
  - The robot encounters a dynamic obstacle along its current path, requiring an updated route to avoid collisions.
  - A timeout condition occurs, ensuring periodic path updates even if no significant changes are detected.

2) How the Tracker Calls the Gridder: The interaction between the `Tracker` and the `Gridder` is implemented through the `Grid2D` interface. The primary method invoked by the `Tracker` is:

```
Result getPaths(TPoint source, TPoint target),
```

where:

- `source` is the robot's current position (x_r, y_r).
- `target` is the detected target's position (x_t, y_t).
- `Result` contains the computed path and metadata, including a validity flag and error messages.

Steps in the Request Process:

1. **Determine Robot and Target Positions:** The robot's current position (x_r, y_r) is retrieved from odometry data, while the target's position (x_t, y_t) is provided by the object detection system.

2. **Invoke the Path-Planning Method:** The `Tracker` invokes `getPaths` on the `Gridder` component, passing (x_r, y_r) as the source and (x_t, y_t) as the target.

3. **Receive and Process the Path:** The `Gridder` returns a `Result` object containing:

- A sequence of waypoints $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$.
- An error message if path computation failed.

4. **Execute the Path:** The `Tracker` processes the path to compute velocity and rotation commands for the robot, which are sent to the actuators.

3) Mathematical Considerations in the Interaction: Path Cost Function:

The computed path minimizes a cost function C, defined as:

$$C = \sum_{i=1}^{n-1} w_{i,i+1},$$

where $w_{i,i+1}$ represents the weight of the edge between waypoints (x_i, y_i) and (x_{i+1}, y_{i+1}). The weight is determined by:

$$w_{i,i+1} = \begin{cases} d_{i,i+1}, & \text{if cells are } \texttt{Free}, \\ \infty, & \text{if cells are } \texttt{Occupied}. \end{cases}$$

Here, $d_{i,i+1} = \|\mathbf{P}_i - \mathbf{P}_{i+1}\|$ is the Euclidean distance between consecutive waypoints.

Synchronization and Concurrency:

To ensure safe interaction, the `Tracker` and `Gridder` components operate with synchronized access to the grid:

- The `Tracker` requests paths only when the `Gridder` is not actively updating the grid.
- A mutex or read-write lock ensures that the grid state is consistent during path computation.

The time complexity of the path request is dominated by Dijkstra's algorithm, which runs in $O((n \times m)\log(n \times m))$ for a grid with $n \times m$ cells.

4) Advantages of the Interaction: This modular interaction between the `Tracker` and the `Gridder` provides several advantages:

- **Real-time performance:** Paths are recalculated dynamically based on the robot's and target's positions, ensuring adaptability to changing environments.
- **Modularity:** The `Tracker` remains agnostic to the specific implementation of the path-planning algorithm in the `Gridder`.
- **Reliability:** By using a well-defined interface, the interaction minimizes the risk of errors and ensures smooth operation.

This interaction enables the system to navigate efficiently and robustly, even in dynamic and unpredictable environments.

H. Handling Scenarios with No Available Path

In dynamic environments, it is possible that the robot may encounter situations where no valid path can be computed to the target due to obstacles or restrictions in the grid. This section explains the behavior of the system in such scenarios and the mechanisms implemented to ensure robustness.

1) Detection of No Available Path: The absence of an available path is identified during the execution of the `dijkstraAlgorithm` method in the `Gridder` component. This occurs under the following conditions:

- **Target Unreachable:** If the target cell (i_t, j_t) is marked as `Occupied`, the target is considered unreachable.
- **Disconnected Regions:** If there is no sequence of `Free` cells connecting the robot's position to the target, the algorithm cannot compute a valid path.
- **Timeout in Search:** If the algorithm exhausts all possible nodes without reaching the target, it determines that no path exists.

Mathematically, the absence of a path is expressed as:

$$\text{Path} = \varnothing, \quad \text{if } \forall (i,j) \in \text{Grid}, C(i,j) = \infty,$$

where $C(i,j)$ is the cumulative cost to reach cell (i,j), and ∞ indicates that the cell is unreachable.

2) System Behavior When No Path Is Available: When no path is available, the system transitions into a fail-safe state to handle the situation gracefully. The specific behaviors are as follows:

1. Tracker Response:

The `ObsPersonTracker` component evaluates the result of the path request. If the `Result.valid` flag returned by the `Gridder` is set to `false`, the tracker:

- Stops the robot's movement by issuing a command to set the velocity and rotational speed to zero:

$$\text{Velocity} = 0, \quad \text{Rotation} = 0.$$

- Displays a warning in the user interface indicating that the target is unreachable.

This behavior ensures that the robot does not proceed with incomplete or invalid navigation data.

2. Recalculation Attempts:

The system periodically attempts to recalculate the path. This is achieved by:

- Continuously monitoring the grid for changes, such as the removal of obstacles or new target positions.
- Triggering a new path-planning request at fixed intervals Δt, where:

$$\Delta t \approx \frac{1}{f_{\text{update}}},$$

and f_{update} is the update frequency of the `update_grid` method.

3. Search Behavior:

If the recalculation attempts fail, the robot transitions into a `Search` state, as implemented in the `state_machine` method of the `Tracker`. In this state:

- The robot rotates on the spot to rescan the environment with its LiDAR.
- The `update_grid` method continuously processes new sensor data, attempting to identify new `Free` cells or alternative paths.

The mathematical condition for exiting the `Search` state is:

$$\exists \text{Path} \neq \varnothing, \quad \text{where the target becomes reachable.}$$

3) Mathematical Analysis of Timeout Handling: Timeout conditions in the path-planning process are designed to ensure responsiveness. The algorithm terminates if the number of explored nodes exceeds a threshold N_{max}:

$$N_{\text{max}} = \alpha \cdot n \cdot m,$$

where:

- n and m are the grid dimensions.
- α is a scaling factor that depends on the density of obstacles.

By limiting the search space, the system avoids excessive delays in detecting unreachable targets.

4) Advantages of the Fail-Safe Mechanisms: The fail-safe mechanisms implemented in the system provide the following benefits:

- **Safety:** The robot halts immediately upon detecting that no valid path exists, avoiding unintended collisions or unsafe movements.
- **Resilience:** The system actively monitors the environment and retries path planning, ensuring adaptability to dynamic changes.
- **Efficiency:** By using timeout mechanisms and recalculation intervals, the system maintains real-time performance without excessive computational overhead.

These mechanisms ensure that the robot can handle scenarios with no available path gracefully, maintaining operational robustness in dynamic and cluttered environments.

I. Controller for Path Following

The controller is a critical component responsible for translating the computed path into actionable motion commands, allowing the robot to navigate towards its target efficiently and safely. This section explains the design and implementation of the controller, referencing its integration with the `Gridder` and `ObsPersonTracker` components.

1) Overview of the Controller: The controller operates as part of the `ObsPersonTracker` component, executing the following key tasks:

- Receives a sequence of waypoints $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ from the `Gridder`.
- Computes linear velocity v and angular velocity ω commands based on the robot's current pose (x_r, y_r, θ_r) and the next waypoint.
- Handles obstacle avoidance dynamically if deviations from the path are required.

2) Path Following Algorithm: The controller uses a proportional controller to compute motion commands. For each waypoint (x_t, y_t), the control law is based on the relative position of the target with respect to the robot's frame:

$$x'_t = x_t - x_r, \quad y'_t = y_t - y_r,$$

where (x_r, y_r) is the robot's current position. The target angle θ_t and distance d_t are computed as:

$$\theta_t = \text{atan2}(y'_t, x'_t), \quad d_t = \sqrt{x'^2_t + y'^2_t}.$$

The control outputs are then defined as:

$$v = K_v \cdot d_t, \quad \omega = K_\theta \cdot (\theta_t - \theta_r),$$

where:

- K_v and K_θ are proportional gains for linear and angular velocities, respectively.
- θ_r is the robot's current orientation.

Dynamic Path Recalibration:

If the robot encounters an obstacle along the current path, the controller halts the robot and requests a new path from the `Gridder`. The system ensures safety by setting:

$$v = 0, \quad \omega = 0,$$

until a new path is available.

3) Handling Waypoints and Target Switching: The controller processes the path sequentially:

- The robot moves towards the first waypoint (x_1, y_1). When $d_t < \epsilon$, where ϵ is a small threshold, the controller switches to the next waypoint.
- The process continues until the final waypoint (x_n, y_n) is reached, satisfying:

$$d_t \leq \epsilon_{\text{final}}, \quad |\theta_t - \theta_r| \leq \epsilon_\theta.$$

4) Mathematical Considerations: Stability of the Controller:

The proportional control law ensures asymptotic convergence to the target under the following conditions:

- $K_v > 0$: Positive gain ensures forward motion proportional to the distance.
- $K_\theta > 0$: Positive gain corrects angular deviations.
- $\theta_t - \theta_r \in (-\pi, \pi)$: Angular error is normalized to avoid discontinuities.

The convergence is guaranteed as long as the path remains obstacle-free and the robot operates within its dynamic limits.

Velocity and Acceleration Constraints:

To ensure smooth motion and avoid instability, velocity and acceleration constraints are applied:

$$v \leq v_{\max}, \quad |\dot{\omega}| \leq \alpha,$$

where $v_{\max}$ is the maximum allowable velocity, and α is the maximum angular acceleration.

5) Integration with the System: The controller interacts with the `Gridder` and the robot's actuators as follows:

- **Path Input:** Receives the computed path from the `Gridder`.
- **State Feedback:** Continuously updates the robot's position and orientation using odometry or other sensors.
- **Command Output:** Sends motion commands to the robot's actuators via the omnirobot proxy:

  $$\text{setSpeedBase}(v_x, v_y, \omega),$$

  where v_x and v_y are the Cartesian components of v, derived as:

  $$v_x = v\cos(\theta_r), \quad v_y = v\sin(\theta_r).$$

6) Failure Handling: If the robot fails to converge to the target within a timeout period, the controller:

- Halts the robot and transitions to the `WAIT` state in the `state_machine`.
- Requests a new path from the `Gridder` to attempt recovery.

7) Advantages of the Controller Design: The controller provides the following benefits:

- **Robustness:** It ensures safe and reliable navigation, even in dynamic environments.
- **Flexibility:** Proportional gains can be tuned for different robot dynamics and environments.
- **Real-time Performance:** The controller operates efficiently, allowing real-time adjustments to the robot's motion.

This controller ensures smooth and efficient navigation along the planned path, while dynamically adapting to environmental changes and maintaining safety.

J. Conclusions

The proposed robotic navigation system successfully integrates a grid-based environment representation, a path-planning module, and a motion controller to navigate reliably in dynamic environments. This work highlights the mathematical foundations and key design decisions, addressing critical challenges to ensure robust performance.

1) Key Contributions: The main contributions of this work are:

- **Grid-Based Representation:** A 2D grid dynamically updated using LiDAR data ensures accurate environment mapping and scalable navigation.
- **Path-Planning Algorithm:** Dijkstra's algorithm guarantees collision-free paths, leveraging a modular interface between components.
- **Motion Control:** A proportional controller translates paths into smooth motion commands, respecting velocity and acceleration constraints.

2) Challenges and Solutions: Throughout the implementation, the following challenges were encountered and addressed:

- **Concurrency Issues:** Managed using mutexes and double buffering to ensure consistent data access and avoid race conditions.
- **Unreachable Paths:** Handled with fail-safe states, dynamic path recalculation, and adaptive responses to environmental changes.
- **Dynamic Obstacles:** Resolved using obstacle expansion techniques and real-time path recalculation for efficient avoidance.
- **Path Ambiguity in Obstacle Avoidance:** Indecisive behavior due to equally optimal paths was identified but remains unresolved, highlighting the need for enhanced decision-making in path planning.

3) Future Work: Future efforts should focus on:

- Extending the system to 3D environments for navigation in multi-level spaces.
- Exploring advanced path-planning algorithms such as A* or RRT for complex scenarios.
- Integrating learning-based approaches to optimize navigation through adaptive decision-making.

4) Final Remarks: This work lays a solid foundation for autonomous navigation, addressing real-time adaptability and challenges in dynamic environments. While the system performs effectively, future advancements in decision-making and scalability are necessary for even greater robustness.

REFERENCES

[1] Davide Brugali and Patrizia Scandurra, Component-Based Robotic Engineering (Part I) Reusable Building Blocks

[2] Davide Brugali and Azamat Shakhimardanov, Component-Based Robotic Engineering (Part II) Systems and Models

[3] R. R. Murphy, Introduction to AI Robotics, 2nd ed. Cambridge, MA, USA: MIT Press, 2019.

[4] P. Corke, Robotics, Vision and Control: Fundamental Algorithms in MATLAB, 2nd ed. Berlin, Germany: Springer, 2017.

AUTHORS



Ismael Díaz Romero



José Javier Pérez Sánchez



Sergio Jiménez Macías